Standalone Mail → Telegram App — документация по фактической реализации

Что это за документ

Это не архитектурный план и не ТЗ. Это описание того, что **реально реализовано** в проекте `/opt/hermes/mail-telegram-app` на момент проверки кода и запуска тестов.

Дата проверки: 2026-04-22 Проект: `mail-telegram-app` Статус тестов: **28 passed**

Краткий вывод

На текущем этапе это уже рабочий standalone backend-прототип, который умеет:

- поднимать FastAPI-приложение;
- инициализировать SQLite-базу;
- создавать и хранить alias-адреса;
- принимать Telegram webhook updates;
- обрабатывать Telegram-команды `/start` и кнопку `Сгенерировать почту`;
- опрашивать Gmail API;
- забирать письма по Gmail message id;
- извлекать alias из заголовков письма;
- сохранять входящее письмо в БД;
- маршрутизировать письмо в Telegram по таблице alias -> chat_id;
- отправлять письмо в Telegram;
- хранить delivery records;
- хранить служебные курсоры worker-процессов в `app_settings`.

Это уже не пустой scaffold. Основная вертикаль `Gmail -> parse -> DB -> route -> Telegram` реализована.

Что проверено при подготовке документа

Проверка выполнена по исходному коду и тестам:

- просмотрены ключевые модули `app/`, `app/api/`, `app/services/`, `app/repositories/`;

- просмотрены тесты `tests/`;
- выполнен `pytest -q`;
- проверено наличие локальной SQLite БД `data/app.db`.

Фактическая структура проекта

Основные файлы

- `app/main.py` — FastAPI bootstrap и выбор режима запуска;
- `app/config.py` — загрузка конфигурации из environment и локального `.env`;
- `app/db.py` — инициализация SQLite и создание таблиц;
- `app/worker.py` — polling worker для Gmail и Telegram updates;
- `app/api/aliases.py` — HTTP API для alias-ов;
- `app/api/telegram.py` — Telegram webhook endpoint;
- `app/services/alias_service.py` — генерация alias local part и нормализация email;
- `app/services/bot_service.py` — Telegram bot UX: `/start`, кнопка и создание alias;
- `app/services/gmail_client.py` — работа с Gmail API и refresh access token;
- `app/services/parser_service.py` — разбор заголовков и декодирование Gmail body;
- `app/services/routing_service.py` — alias -> telegram_chat_id;
- `app/services/ingestion_service.py` — сохранение входящего письма и доставка в Telegram;
- `app/services/telegram_client.py` — обертка над Telegram Bot API;
- `app/repositories/*.py` — доступ к SQLite.

Реальный масштаб кода

По фактическому подсчету в проекте: - Python-файлов в app/: **23** - Python-файлов тестов: **12** - строк Python-кода в app/: **1114** - строк Python-кода в тестах: **859**

Конфигурация

Модуль: `app/config.py`

Реализован класс `AppSettings` со следующими полями:

- `app_env`
- `database_url`
- `gmail_client_secret_path`

- gmail_token_path
- gmail_poll_seconds
- default_from_alias_domain
- telegram_bot_token

Источник конфигурации

1. Environment variables.
2. Локальный файл .env в корне проекта.

Локальный .env автоматически подгружается, если: - не выставлен APP_ENV=test; - не идет запуск под pytest.

Это важно: тесты изолированы от локального .env.

Пример ожидаемых переменных

Из .env.example: - APP_ENV - DATABASE_URL - GMAIL_CLIENT_SECRET_PATH - GMAIL_TOKEN_PATH - GMAIL_POLL_SECONDS - DEFAULT_FROM_ALIAS_DOMAIN - TELEGRAM_BOT_TOKEN

FastAPI приложение

Модуль: app/main.py

Реализовано: - создание FastAPI(title="mail-telegram-app"); - подключение роутеров alias API и Telegram webhook API; - health endpoint GET /health.

Health endpoint

GET /health

Возвращает:

```
{"status": "ok", "service": "mail-telegram-app"}
```

Режимы запуска

Через python -m app.main --mode ... поддержаны режимы: - api - worker - all-in-one

По факту в __main__ сейчас реализовано следующее поведение: - worker -> запускается run_worker_loop(); - api и all-in-one -> вызывается create_app() и печатается bootstrap message.

Важно: полноценный HTTP server из main.py сам не стартует. Для реального API запуска нужен внешний ASGI runner, например uvicorn.

База данных

Модуль: `app/db.py`

Используется только SQLite URL формата `sqlite:///...`. Если URL не sqlite, код явно выбрасывает ошибку: - `ValueError("Only sqlite URLs are supported in v1")`

Инициализация БД

`initialize_database(settings)`: - вычисляет путь к sqlite-файлу; - создает директорию при необходимости; - выполняет SQL schema bootstrap.

Фактически созданные таблицы

1. `aliases`
2. `inbound_messages`
3. `deliveries`
4. `app_settings`

Таблица `aliases`

Поля: - `id` - `alias_local_part` - `alias_email` (unique) - `telegram_chat_id` - `telegram_user_id` - `description` - `is_active` - `created_at` - `updated_at`

Таблица `inbound_messages`

Поля: - `id` - `gmail_message_id` (unique) - `internet_message_id` - `thread_id` - `alias_email` - `delivered_to` - `from_address` - `subject` - `received_at` - `body_text` - `body_html` - `raw_headers_json` - `parse_status` - `created_at`

Таблица `deliveries`

Поля: - `id` - `inbound_message_id` - `telegram_chat_id` - `telegram_message_id` - `status` - `error_text` - `delivered_at` - `created_at`

Таблица `app_settings`

Поля: - `id` - key (unique) - `value` - `updated_at`

Repository layer

`AliasRepository`

Реализованы методы: - `create_alias(...)` - `get_by_alias_email(...)` - `list_aliases()`

Назначение: - сохраняет alias; - ищет маршрут по `alias_email`; - отдает список alias-ов для API.

InboundMessageRepository

Реализованы методы: - `get_by_gmail_message_id(...)` - `update_parse_status(...)` - `create_message(payload)`

Особенность: - есть дедупликация по `gmail_message_id`; - повторное создание того же письма возвращает существующий id.

DeliveryRepository

Реализован метод: - `create_delivery(...)`

Назначение: - журнал доставки письма в Telegram.

AppSettingsRepository

Реализованы методы: - `get(key)` - `set(key, value)`

Использование по факту: - хранение курсора Gmail worker; - хранение курсора Telegram polling.

Alias management

Генерация alias

Модуль: `app/services/alias_service.py`

Реализовано: - `generate_alias_local_part(length=8)` - `normalize_alias_email(local_part, domain)`

Свойства: - local part строится из `a-z0-9`; - используется `secrets.choice`, то есть генерация криптографически случайная; - email нормализуется к lowercase.

HTTP API alias-ов

Модуль: `app/api/aliases.py`

`POST /aliases`

Создает alias.

Входная схема (`AliasCreateRequest`): - `telegram_chat_id` — обязательно; - `description` — опционально; - `telegram_user_id` — опционально; - `local_part_length` — по умолчанию 8, диапазон 4..32.

Поведение: - если не задан `DEFAULT_FROM_ALIAS_DOMAIN`, возвращается 500; - до 10 попыток генерации уникального alias; - alias сохраняется в SQLite; - ответ возвращается в схеме `AliasResponse`.

`GET /aliases`

Возвращает все alias-ы из БД списком.

Telegram integration

Telegram Bot API client

Модуль: app/services/telegram_client.py

Реализованы методы: - send_message(chat_id, text, reply_markup=None) - get_updates(offset=None, timeout=0) - delete_webhook(drop_pending_updates=False) - get_webhook_info() - get_me()

Особенности: - используется прямой вызов https://api.telegram.org/bot<TOKEN>/<method>; - transport реализован через httpx.post(...); - preview ссылок отключается через disable_web_page_preview=True.

Telegram webhook endpoint

Модуль: app/api/telegram.py

Реализован endpoint: - POST /telegram/webhook

Поведение: - принимает сырой Telegram update как dict; - берет AppSettings и AliasRepository через dependency injection; - создает TelegramClient, если он не был инжектирован снаружи; - вызывает handle_update(...); - всегда возвращает {"ok": True}.

Логика Telegram-бота

Модуль: app/services/bot_service.py

Реализовано: - reply keyboard с кнопкой Сгенерировать почту; - обработка /start; - обработка текста Сгенерировать почту; - обработка всех прочих текстов через подсказку использовать кнопку.

/start

Бот отправляет приветствие и клавиатуру.

Сгенерировать почту

Бот: - генерирует alias; - связывает его с текущим chat_id; - сохраняет также telegram_user_id; - возвращает пользователю email-адрес.

Неизвестная команда

Бот отвечает текстом-подсказкой и повторно показывает клавиатуру.

Gmail integration

Gmail client

Модуль: app/services/gmail_client.py

Реализовано: - чтение токена из JSON файла; - fallback на поля `access_token` или `token`; - refresh access token через `refresh_token`; - чтение client credentials из отдельного JSON; - retry после `401 Unauthorized`; - получение списка message ids; - получение полного письма в формате `full`.

Реализованные методы

- `_load_token_payload()`
- `_write_token_payload(payload)`
- `_load_access_token()`
- `refresh_access_token()`
- `_headers()`
- `_get_with_auto_refresh(url, params)`
- `list_message_ids(...)`
- `get_message(message_id)`

Запросы к Gmail API

Используется endpoint: - `https://gmail.googleapis.com/gmail/v1/users/me/messages`

По факту реализованы операции: - `GET /messages?q=in:inbox&maxResults=...` - `GET /messages/{id}?format=full`

Парсинг входящих писем

Модуль: `app/services/parser_service.py`

Реализовано: - `flatten_headers(...)` - `extract_delivered_alias(headers)` - `decode_gmail_body(data)`

Извлечение alias

Приоритет источников alias: 1. `X-Forwarded-For` — если там есть email, отличный от `@gmail.com`; 2. `Delivered-To` 3. `X-Original-To` 4. `To`

Поведение: - email нормализуется к lowercase; - используется regex fallback, если стандартный parseaddr не сработал.

Декодирование body

Поддержано: - base64url-decoding тела письма; - добавление padding для корректного декодирования; - `utf-8` decode с `errors="replace"`.

Ограничение текущей реализации: - берется только `payload.body.data`; - multipart traversal и выбор лучшей text/plain/text/html части пока не реализованы.

Routing и ingestion

Routing

Модуль: app/services/routing_service.py

Реализовано: - resolve_chat_id_for_alias(repo, alias_email)

Логика: - ищет alias в таблице aliases; - если alias не найден -> None; - если alias неактивен -> None; - иначе возвращает telegram_chat_id.

Ingestion

Модуль: app/services/ingestion_service.py

Реализован основной сценарий ingest_gmail_message(...).

Что делает функция: 1. Выравнивает заголовки письма. 2. Извлекает alias из заголовков. 3. Формирует payload для inbound_messages. 4. Сохраняет письмо в БД. 5. Если alias не найден -> статус missing-alias. 6. Если alias найден, но маршрут не найден -> статус no-route. 7. Если маршрут найден -> формирует текст для Telegram. 8. Отправляет сообщение в Telegram. 9. Создает запись в deliveries со статусом delivered.

Формат Telegram-сообщения

Фактический шаблон: - New email for <alias_email> - From: <from_address> - Subject: <subject> - пустая строка - body письма

Статусы обработки

По коду реально используются: - parsed - missing-alias - no-route - delivered

Worker

Модуль: app/worker.py

Реализованы три функции: - process_once(...) - process_telegram_updates_once(...) - run_worker_loop()

process_once(...)

Назначение: - одна итерация обработки входящих Gmail сообщений.

Что делает: 1. Инициализирует БД. 2. Создает Gmail client и Telegram client при необходимости. 3. Получает список последних Gmail message ids. 4. Читает из app_settings курсор last_seen_gmail_message_id. 5. Отбирает только новые письма до первой уже известной границы. 6. Обрабатывает новые письма в хронологическом порядке. 7. Сохраняет новый cursor в app_settings. 8. Возвращает статистику обработки.

Возвращаемые поля: - processed - delivered - skipped_seen - last_seen_gmail_message_id

`process_telegram_updates_once(...)`

Назначение: - одна итерация polling Telegram updates.

Что делает: 1. Инициализирует БД. 2. Поднимает Telegram client. 3. Читает offset из `app_settings` по ключу `telegram_last_update_id`. 4. Вызывает `get_updates(offset=..., timeout=0)`. 5. На каждый update вызывает `handle_update(...)`. 6. Сохраняет новый `last_update_id`.

Возвращает: - `processed` - `handled` - `last_update_id`

`run_worker_loop()`

Бесконечный цикл: 1. `process_telegram_updates_once(settings)` 2. `process_once(settings)` 3. `sleep(settings.gmail_poll_seconds)`

Итого: в текущем виде worker умеет и опрашивать Gmail, и параллельно подбирать Telegram bot updates через polling.

Покрытие тестами

Фактический результат запуска: - **28 passed in 2.65s**

Что покрыто тестами

По именам и содержанию тестов подтверждается покрытие следующих сценариев:

- bootstrap приложения и health endpoint;
- создание таблиц SQLite;
- загрузка конфигурации;
- alias generation и parser utilities;
- alias API (`POST /aliases`, `GET /aliases`);
- webhook endpoint Telegram;
- bot UX (`/start`, кнопка генерации alias, unknown text);
- Gmail client;
- refresh token logic;
- retry при 401;
- repositories и routing;
- ingestion service;
- worker polling и cursor persistence.

Фактическое состояние локальной БД

Во время проверки обнаружен файл: - `data/app.db`

Содержимое БД на момент проверки: - `aliases`: **21** записей - `inbound_messages`: **22** записей - `deliveries`: **4** записей - `app_settings`: **2** записей

Это подтверждает, что проект уже создавал реальную SQLite-базу, а не только тестовые in-memory структуры.

Реализовано vs не реализовано

Уже реализовано

- standalone структура приложения без зависимости от Hermes runtime API;
- FastAPI bootstrap;
- SQLite persistence;
- alias registry;
- Telegram webhook endpoint;
- Telegram polling support в worker;
- Gmail polling support;
- token refresh для Gmail;
- письмо -> парсинг -> запись в БД -> отправка в Telegram;
- тесты на основной happy path и несколько edge cases.

Пока не реализовано или реализовано частично

- Alembic migrations;
- SQLAlchemy ORM;
- Dockerfile / docker-compose / systemd unit в этом каталоге;
- полноценный production entrypoint для API из `main.py`;
- multipart MIME parsing;
- HTML body extraction и fallback formatting;
- запись delivery ошибок при падении `send_message`;
- admin endpoints;
- disable/enable alias endpoints;
- webhook registration management;
- observability/logging layer;
- AI summary layer.

Практический вывод

Текущая реализация уже годится как работающий standalone prototype / MVP backend.

Это не просто каркас: - есть реальная БД; - есть реальные API endpoints; - есть реальный Gmail client; - есть реальный Telegram client; - есть реальный worker; - есть реальная обработка писем и маршрутизация; - есть проходящий тестовый набор.

Главные технологические долги текущей версии: 1. упрощенный MIME parsing; 2. упрощенный runtime bootstrap API режима; 3. отсутствие production packaging; 4. слабая обработка delivery failures.

Следующий разумный шаг

Если продолжать доводить продукт до production-ready состояния, ближайшая практическая очередь такая:

1. добавить нормальный запуск API через `uvicorn`/entrypoint;
2. добавить systemd unit или Docker packaging;
3. расширить MIME parser на multipart письма;
4. логировать и сохранять delivery failures;
5. добавить alias disable/list/admin операции;
6. провести live end-to-end проверку на реальном письме и реальном Telegram чате.